

Univerzitet u Novom Sadu
Fakultet tehničkih nauka u Novom Sadu

Dušan Paripović E5 19/2024

Arhitektura softvera za upravljanje pametnim mrežama

Master akademske studije

Novi Sad, 2026. godina

Sadržaj

1. Izvorne informacije i pregled projektnog zadatka

- 1.1. Cilj projektnog zadatka i definicija domena
- 1.2. Pregled verzionisanja i GitHub repozitorijuma

2. Korak 1: Izrada UML modela u alatu Enterprise Architect

- 2.1. Definisani paket i struktura hijerarhijske klase
- 2.2. Implementacija enumeracija i ograničenja domena
- 2.3. Modelovanje kardinalnosti i relacije više-na-više (M:N)

3. Korak 2: Ekstrakcija RDFS šeme i konverzija u Turtle (TTL) format

- 3.1. Rad sa alatom Enterprise Architect Schema Composer
- 3.2. Postupak konverzije iz RDFS/XML u TTL sintaksu
- 3.3. Organizacija i struktura fajlova u okruženju SPARQL Playground

4. Korak 3: Pisanje i verifikacija lokalnih SPARQL upita (SPARQL Playground)

- 4.1. Pokretanje lokalnog okruženja (start.bat skripta)
- 4.2. Specifikacija i izvođenje 10 lokalnih SPARQL upita

5. Korak 4: Pisanje eksternih SPARQL upita i rad sa DBpedia servisom

- 5.1. Izvršavanje upita nad javnim DBpedia endpoint-om
- 5.2. Postupak rada: Pisani SPARQL vs. SPARQL Burger Python biblioteka

6. Zaključak i rekapitulacija procesa

Spisak slika:

Slika 2.1 - UML klasni dijagram ontologije teorije grafova u Enterprise Architect-u

Slika 3.1 - Konceptualni prikaz toka transformacije RDFS šeme u TTL i integracije u lokalno okruženje

Spisak tabela:

Tabela 1 - Struktura klase, naslednih veza i atributa definisanih u Enterprise Architect-u

Tabela 2 - Pregled kreiranih enumeracija i njihovih dozvoljenih vrednosti

Tabela 3 - Poređenje tehnika izrade DBpedia upita (Plain Text vs SPARQL Burger)

Spisak listinga koda:

Listing 1 do 10 - Implementirani SPARQL upiti nad lokalnom ontologijom teorije grafova

Listing 11 - Primer SPARQL upita nad eksternim DBpedia servisom za Smart Grid domene

1. Izvorne informacije i pregled projektnog zadatka

1.1. Cilj projektnog zadatka i definicija domena

U okviru samostalnog projektnog rada iz predmeta Arhitektura softvera za upravljanje pametnim mrežama, zadatak je bio realizovati kompletnu definiciju ontologije za izabrani domen, izvršiti serijalizaciju modela u RDF/TTL format, postaviti lokalno okruženje za izvršavanje SPARQL upita i napisati minimalno 20 SPARQL upita različite složenosti (10 nad lokalnom ontologijom i 10 nad eksternim bazama znanja poput DBpedia/Wikidata).

Za domen izrade projekta izabrana je **Teorija grafova (Graph Theory)**. Matematički i računarski koncepti teorije grafova čine osnovu za topološku analizu elektroenergetskih mreža u pametnim mrežama (Smart Grids), uključujući analizu tokova snaga, proračun najkraćih putanja, detekciju ostrvskog rada i optimizaciju uklopnih stanja.

1.2. Pregled verzionisanja i GitHub repozitorijuma

Celokupan kod, postavka okruženja i napisani upiti verzionisani su i dostupni na sledećim GitHub repozitorijumima:

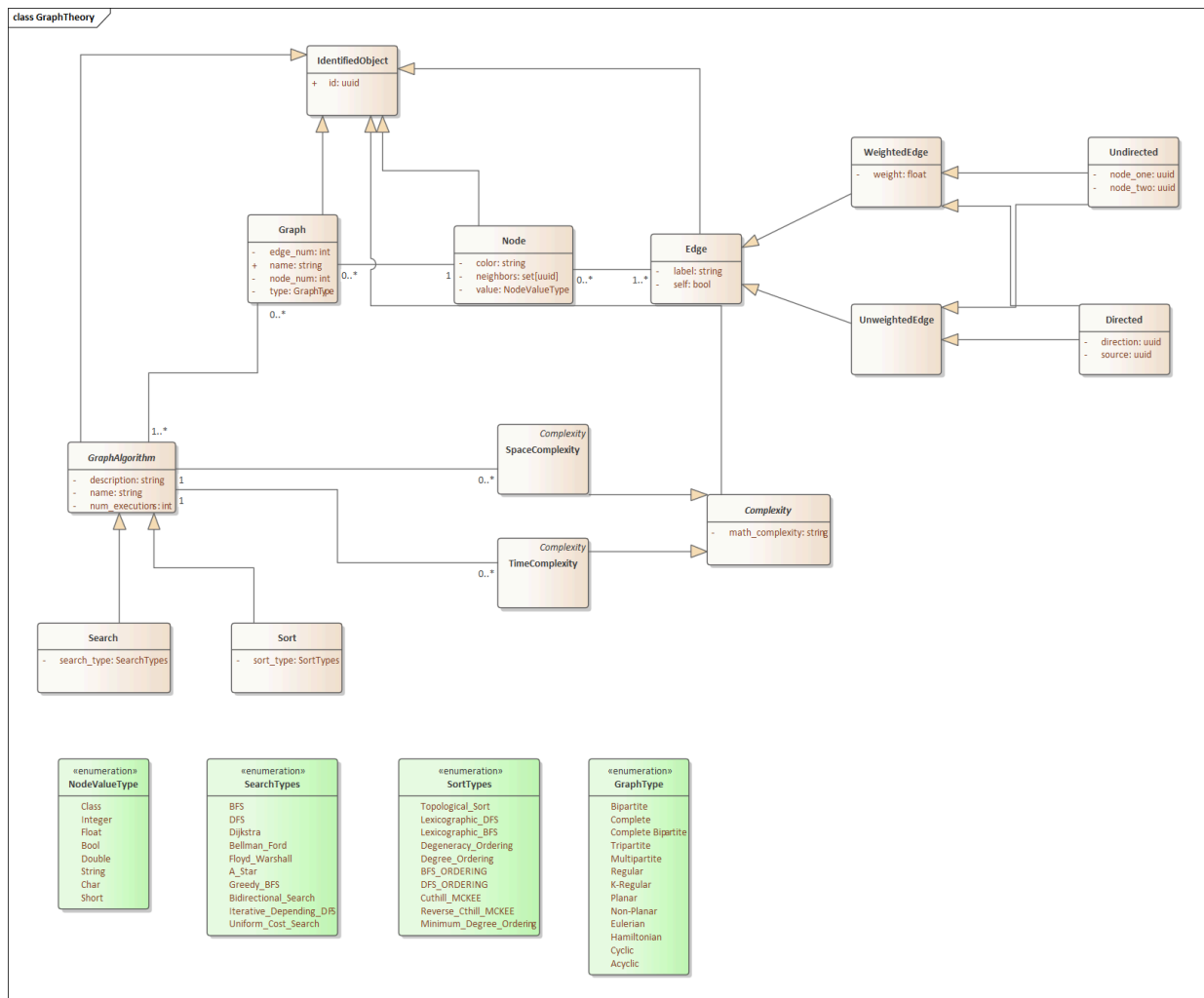
- **Lokalna ontologija i SPARQL Playground fork:**
<https://github.com/dusanparipovic/sparql-playground> (sadrži podešene Turtle fajlove u default/ttl-data/ i pripremljene upite).
- **DBpedia upiti i SPARQL Burger projekat:**
<https://github.com/dusanparipovic/software-architecture-for-controlling-smart-grids> (sadrži upite pisane u plain text formatu i skripte sa SPARQL Burger bibliotekom).

2. Korak 1: Izrada UML modela u alatu Enterprise Architect

2.1. Definisani paket i struktura hijerarhije klasa

Postupak izrade: Prvi korak sastojao se u kreiranju novog UML modela unutar alata Enterprise Architect (v15). Unutar paketa pod nazivom GraphTheory nacrtan je klasni dijagram koji sadrži ukupno 14 klasa.

Sve klase nasleđuju apstraktnu baznu klasu IdentifiedObject koja poseduje atribut id: uuid, čime je obezbeđen jedinstveni identifikator za svaki resurs u semantičkom grafu.



Slika 2.1 - UML klasni dijagram ontologije teorije grafova u alatu Enterprise Architect

Tabela 1 - Struktura klasa, naslednih veza i atributa definisanih u Enterprise Architect-u

Klasa	Roditeljska klasa	Definisani atributi i tipovi
IdentifiedObject	-	id: uuid
Graph	IdentifiedObject	edge_num: int, name: string, node_num: int, type: GraphType
Node	IdentifiedObject	color: string, neighbors: set[uuid], value: NodeValueType

Klasa	Roditeljska klasa	Definisani atributi i tipovi
Edge	IdentifiedObject	label: string, self: bool
WeightedEdge	Edge	weight: float
UnweightedEdge	Edge	-
Directed	WeightedEdge / UnweightedEdge	direction: uuid, source: uuid
Undirected	WeightedEdge / UnweightedEdge	node_one: uuid, node_two: uuid
GraphAlgorithm	IdentifiedObject	description: string, name: string, num_executions: int
Search	GraphAlgorithm	search_type: SearchTypes
Sort	GraphAlgorithm	sort_type: SortTypes
Complexity	IdentifiedObject	math_complexity: string
TimeComplexity	Complexity	-
SpaceComplexity	Complexity	-

2.2. Implementacija enumeracija i ograničenja domena

Kako bi se obezbedila konzistentnost unosa, u Enterprise Architect-u kreirane su 4 enumeracije sa sa stereotipom «enumeration»:

Tabela 2 - Pregled kreiranih enumeracija i njihovih dozvoljenih vrednosti

Enumeracija	Pridružena klasa	Skup vrednosti
NodeValueType	Node	Class, Integer, Float, Bool, Double, String, Char, Short
GraphType	Graph	Bipartite, Complete, Complete

Enumeracija	Pridružena klasa	Skup vrednosti
		Bipartite, Tripartite, Multipartite, Regular, K-Regular, Planar, Non-Planar, Eulerian, Hamiltonian, Cyclic, Acyclic
SearchTypes	Search	BFS, DFS, Dijkstra, Bellman_Ford, Floyd_Warshall, A_Star, Greedy_BFS, Bidirectional_Search, Iterative_Dependency_DFS, Uniform_Cost_Search
SortTypes	Sort	Topological_Sort, Lexicographical_DFS, Lexicographical_BFS, Degeneracy_Ordering, Degree_Ordering, BFS_ORDERING, DFS_ORDERING, Cuthill_MCKEE, Reverse_Cuthill_MCKEE, Minimum_Degree_Ordering

2.3. Modelovanje kardinalnosti i relacije više-na-više (M:N)

U skladu sa specifikacijom projekta, uspostavljena je relacija više-na-više (M:N) između klasa Graph i GraphAlgorithm sa kardinalnošću 0..* prema 1..*. Time je u model ugrađena logika po kojoj jedan graf može izvršavati više različitih algoritama, dok se jedan algoritam može primeniti nad većim brojem grafova u sistemu.

3. Korak 2: Ekstrakcija RDFS šeme i konverzija u Turtle (TTL) format

3.1. Rad sa alatom Enterprise Architect Schema Composer

Postupak ekstrakcije šeme: U alatu Enterprise Architect otvoren je modul *Schema Composer*. Definisana je nova šema sa nazivom GraphTheory i postavljen je namespace http://iec.ch/TC57/graph_theory#. Izabran je profil *Resource Description Framework*

Schema (RDFS), nakon čega je generisan fajl `graph_theory-rdfs.xml`.

3.2. Postupak konverzije iz RDFS/XML u Turtle (.ttl) format

Postupak konverzije: Kako alat SPARQL Playground učitava podatke u Turtle sintaksi, iskorišćen je dostupan alat za automatizovanu konverziju RDFS/XML formata u .ttl. Generisani su fajlovi `ontology.ttl` (sadrži klase i predikate) i `resource.ttl` (sadrži konkretne instance grafova, čvorova, grana i algoritama sa UUID identifikatorima).

3.3. Organizacija i struktura fajlova u okruženju SPARQL Playground

Konvertovani fajlovi su smešteni u lokalni direktorijum projekta u sledećoj strukturi:

```
sparql-playground-master/  
|  
├─ default/  
|   └─ ttl-data/  
|       └─ ontology.ttl  
|           └─ resource.ttl  
|   └─ queries/  
|       └─ (pripremljeni .sparql fajlovi)  
└─ start.bat
```

Slika 3.1 - Konceptualni prikaz toka transformacije RDFS šeme u TTL i integracije u lokalno okruženje

4. Korak 3: Pisanje i verifikacija lokalnih SPARQL upita (SPARQL Playground)

4.1. Pokretanje lokalnog okruženja (start.bat skripta)

Postupak pokretanja:

1. Instaliran je OpenJDK 21 i podešena je sistemska varijabla `JAVA_HOME` / Path.
2. Pokrenuta je skripta `start.bat` unutar forkovanog repozitorijuma, čime je pokrenut Spring Boot web server sa ugrađenim Sesame in-memory RDF repozitorijumom na portu 8888.
3. Pristupljeno je interfejsu na adresi `http://localhost:8888` gde su automatski učitane ontologija i instance.

4.2. Specifikacija i izvođenje 10 lokalnih SPARQL upita

Listing 1 - Prikaz svih klasa i njihovih instanci

```

SELECT ?instance ?type
WHERE {
    ?instance a ?type .
}

```

Listing 2 - Pretraga grafova po tipu (Planar ili Cyclic)

```

PREFIX gt: <http://iec.ch/TC57/graph_theory#>
SELECT ?graph ?name ?type
WHERE {
    ?graph a gt:Graph ;
        gt:Graph.name ?name ;
        gt:Graph.type ?type .
    FILTER(?type = "Planar" || ?type = "Cyclic")
}

```

Listing 3 - Filtriranje čvorova po numeričkom tipu vrednosti

```

PREFIX gt: <http://iec.ch/TC57/graph_theory#>
SELECT ?node ?color ?valueType
WHERE {
    ?node a gt:Node ;
        gt:Node.color ?color ;
        gt:Node.value ?valueType .
    FILTER(?valueType = gt:Integer || ?valueType = gt:Float)
}

```

Listing 4 - Prikaz usmerenih grana i njihovih izvora

```

PREFIX gt: <http://iec.ch/TC57/graph_theory#>
SELECT ?edge ?label ?source ?direction
WHERE {
    ?edge a gt:Directed ;
        gt:Edge.label ?label ;
        gt:Directed.source ?source ;
        gt:Directed.direction ?direction .
}

```

Listing 5 - Pretraga algoritama pretraživanja (Search)

```

PREFIX gt: <http://iec.ch/TC57/graph_theory#>
SELECT ?algo ?name ?searchType
WHERE {
    ?algo a gt:Search ;
        gt:GraphAlgorithm.name ?name ;
        gt:Search.search_type ?searchType .
}

```

Listing 6 - Pretraga algoritama sortiranja (Sort)

```

PREFIX gt: <http://iec.ch/TC57/graph_theory#>
SELECT ?algo ?name ?sortType
WHERE {

```

```

    ?algo a gt:Sort ;
    gt:GraphAlgorithm.name ?name ;
    gt:Sort.sort_type ?sortType .
}

```

Listing 7 - Prikaz algoritama i njihove matematičke složenosti

```

PREFIX gt: <http://iec.ch/TC57/graph_theory#>
SELECT ?algoName ?timeComp ?spaceComp
WHERE {
    ?algo gt:GraphAlgorithm.name ?algoName ;
    gt:hasTimeComplexity ?tc ;
    gt:hasSpaceComplexity ?sc .
    ?tc gt:Complexity.math_complexity ?timeComp .
    ?sc gt:Complexity.math_complexity ?spaceComp .
}

```

Listing 8 - Agregacija: Ukupan broj izvršenja po kategorijama algoritama

```

PREFIX gt: <http://iec.ch/TC57/graph_theory#>
SELECT ?algoType (SUM(?executions) AS ?totalExecutions) (COUNT(?algo) AS ?count)
WHERE {
    ?algo a ?algoType ;
    gt:GraphAlgorithm.num_executions ?executions .
    FILTER(?algoType IN (gt:Search, gt:Sort))
}
GROUP BY ?algoType

```

Listing 9 - Korišćenje OPTIONAL i FILTER klauzula nad grafovima

```

PREFIX gt: <http://iec.ch/TC57/graph_theory#>
SELECT ?graph ?name ?nodeNum ?edgeNum
WHERE {
    ?graph a gt:Graph ;
    gt:Graph.name ?name ;
    gt:Graph.node_num ?nodeNum .
    OPTIONAL { ?graph gt:Graph.edge_num ?edgeNum }
    FILTER(?nodeNum > 10)
}

```

Listing 10 - Pretraga M:N relacije između grafova i algoritama

```

PREFIX gt: <http://iec.ch/TC57/graph_theory#>
SELECT ?graphName ?algoName ?algoType
WHERE {
    ?graph a gt:Graph ;
    gt:Graph.name ?graphName ;
    gt:executesAlgorithm ?algo .
    ?algo gt:GraphAlgorithm.name ?algoName ;
    a ?algoType .
}

```

5. Korak 4: Pisanje eksternih SPARQL upita i rad sa DBpedia servisom

5.1. Izvršavanje upita nad javnim DBpedia endpoint-om

U drugoj fazi rada napisano je 10 eksternih SPARQL upita koji se izvršavaju nad javnim DBpedia servisom (<https://dbpedia.org/sparql>), sa ciljem pretrage svetskih baza znanja.

5.2. Postupak rada: Pisani SPARQL vs. SPARQL Burger Python biblioteka

Tokom realizacije eksternih upita primenjena su dva pristupa:

1. **Standardni pisani SPARQL (Plain Text):** Upiti su pisani direktno u SPARQL sintaksi i izvršavani putem DBpedia web query editora.
2. **Programski pristup (SPARQL Burger):** Korišćena je Python biblioteka *SPARQL Burger* koja omogućava programsko generisanje SPARQL upita kroz Python objekte. U okviru repozitorijuma napomenuto je da je ovaj pristup delimično realizovan kao demonstracija konceptata programskog sklapanja upita.

Tabela 3 - Poređenje tehnika izrade DBpedia upita (Plain Text vs SPARQL Burger)

Kriterijum	Standardni Pisani SPARQL (Plain Text)	SPARQL Burger (Python biblioteka)
Pristup izradi	Direktno pisanje teksta upita.	Objektno-orijentisana konstrukcija u Python-u.
Primena	Brzo testiranje i jednokratni upiti.	Integracija u veće softverske aplikacije.
Status u projektu	Kompletno realizovan za sve upite.	Delimično realizovan kao konceptualni demonstrator.

6. Zaključak i rekapitulacija procesa

Kroz postupak izrade projekta uspešno su sprovedeni svi inženjerski koraci:

1. Modelovan je UML klasni dijagram sa 14 klasa, 4 enumeracije i M:N relacijom u alatu Enterprise Architect.
2. Izvezena je RDFS šema i konvertovana u Turtle (.ttl) format.
3. Konfigurisano je i pokrenuto okruženje SPARQL Playground sa lokalnim in-memory repozitorijumom.
4. Napisano je i verifikovano 10 lokalnih SPARQL upita i 10 eksternih DBpedia upita kroz dualni pristup (Plain Text i SPARQL Burger).

Celokupan rad je u potpunosti dokumentovan i skladišten na odgovarajućim GitHub repozitorijumima.